word2vec 원리 설명 문서 (Word2Vec Principles Documentation)

📚 문서 네비게이션 (Document Navigation)

| 이전 문서 | 현재 문서 | 다음 문서 |

|----------|----------|----------|

| 📖 README | **1. word2vec 원리 설명** | 2. 구현체 구조 분석 → |

관련 문서:

- 환경 설정 가이드 - 실행 환경 구성
- 활용 사례 및 확장 - 실제 사용 예제

목차 (Table of Contents)

1. word2vec 개요
2. 수학적 배경
3. Skip-gram과 CBOW 모델
4. 계층적 소프트맥스
5. 네거티브 샘플링
6. 벡터 공간의 의미적 관계
7. 아날로지와 벡터 연산

word2vec 개요 (Overview)

word2vec은 2013년 Google의 토마스 미콜로프(Tomas Mikolov)가 제안한 단어 임베딩(Word Embedding) 기법입니다. 이 알고리즘은 텍스트 코퍼스(Text Corpus)에서 단어들을 고차원 벡터 공간(High-dimensional Vector Space)에 매핑하여, 단어 간의 의미적 관계를 수치적으로 표현할 수 있게 합니다.

핵심 아이디어 (Core Idea)

word2vec의 핵심 아이디어는 **"비슷한 맥락에서 나타나는 단어들은 비슷한 의미를 가진다"**는 분포 가설(Distributional Hypothesis)에 기반합니다.

한국어 예시:

- "고양이는 **털이** 부드럽다"
- "강아지는 **털이** 부드럽다"

위 문장에서 "고양이"와 "강아지"는 비슷한 맥락에서 사용되므로, 벡터 공간에서도 가까운 위치에 배치됩니다.

직관적 비유: 마치 친구들을 성격으로 분류하는 것과 같습니다. 비슷한 상황에서 비슷하게 행동하는 친구들은 성격이 비슷하다고 판단하는 것처럼, word2vec도 비슷한 문맥에서 나타나는 단어들을 비슷한 의미로 학습합니다.

수학적 배경 (Mathematical Background)

원-핫 인코딩의 한계 (Limitations of One-hot Encoding)

전통적인 단어 표현 방식인 원-핫 인코딩(One-hot Encoding)은 다음과 같은 문제점을 가집니다:

```
어휘 크기가 V인 경우:

"고양이" = [1, 0, 0, 0, ..., 0] (V차원)

"강아지" = [0, 1, 0, 0, ..., 0] (V차원)
```

문제점:

1. **희소성(Sparsity)**: 대부분의 값이 0으로 메모리 낭비
2. **차원의 저주(Curse of Dimensionality)**: 어휘 크기에 비례하여 차원 증가
3. **의미적 관계 부재**: 단어 간 유사도를 계산할 수 없음

한국어 비유: 원-핫 인코딩은 마치 각 사람을 하나의 번호로만 표현하는 것과 같습니다. "김철수=1번, 이영희=2번"으로 표현하면 두 사람이 얼마나 비슷한지 알 수 없습니다.

word2vec의 해결책 (Word2Vec Solution)

word2vec은 단어를 **밀집 벡터(Dense Vector)**로 표현합니다:

```
"고양이" = [0.2, -0.1, 0.8, ..., 0.3] (d차원, 보통 100~300)

"강아지" = [0.3, -0.2, 0.7, ..., 0.4] (d차원)
```

장점:

1. **밀집성(Density)**: 모든 차원이 의미있는 값을 가짐
2. **효율성(Efficiency)**: 고정된 낮은 차원 (100~300차원)
3. **의미적 유사성(Semantic Similarity)**: 코사인 유사도(Cosine Similarity)로 측정 가능

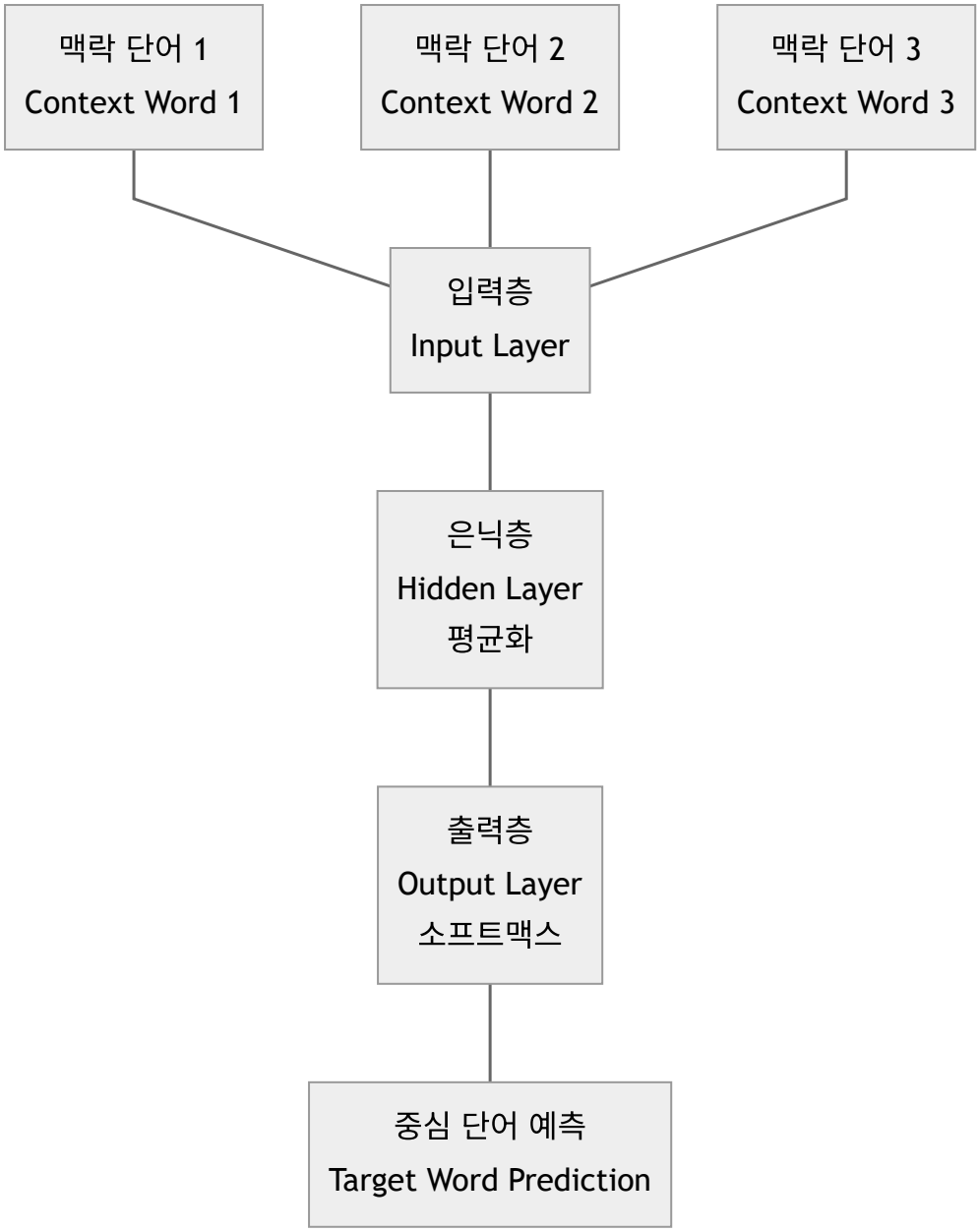
한국어 비유: 이제 각 사람을 여러 특성으로 표현합니다. "김철수=[키:0.8, 성격:0.3, 취미:0.5]"처럼 다차원으로 표현하면 사람들 간의 유사도를 계산할 수 있습니다.

Skip-gram과 CBOW 모델 (Skip-gram and CBOW Models)

word2vec은 두 가지 주요 아키텍처를 제공합니다: **Skip-gram**과 **CBOW(Continuous Bag of Words)**

CBOW (Continuous Bag of Words) 모델

CBOW는 **주변 단어들로부터 중심 단어를 예측**하는 모델입니다.



한국어 예시:

- 입력: "고양이는", "매우", "귀엽다"
- 출력: "**정말**" (중심 단어)

수학적 표현:

```
P(w_t | w_{t-c}, ..., w_{t-1}, w_{t+1}, ..., w_{t+c})
```

여기서:

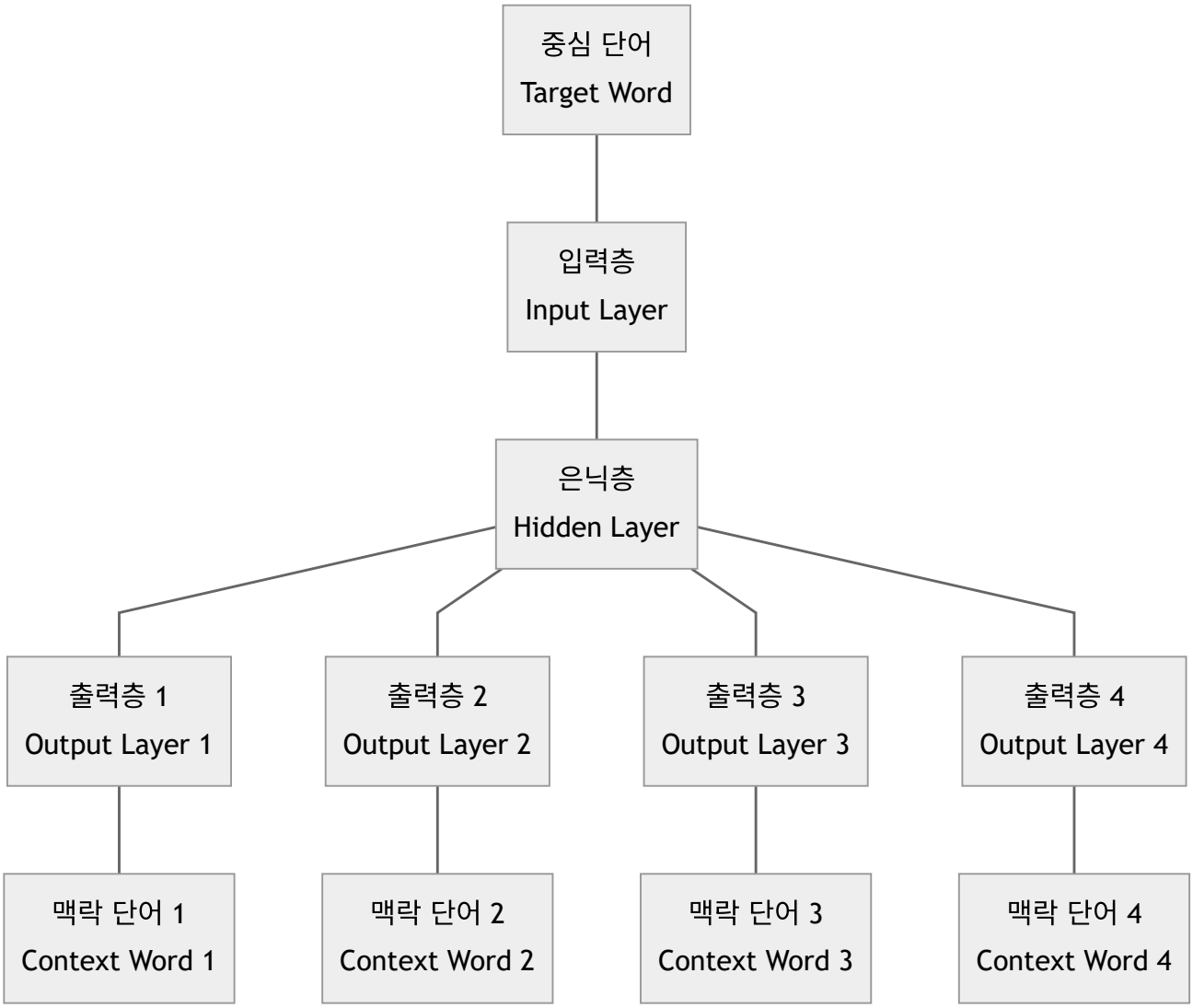
- `w_t` : 중심 단어 (target word)

- `w_{t-c}, ..., w_{t+c}` : 맥락 단어들 (context words)
- `c` : 윈도우 크기 (window size)

직관적 설명: CBOW는 마치 빈칸 맞추기 게임과 같습니다. "고양이는 ___ 귀엽다"에서 빈칸에 들어갈 적절한 단어를 주변 맥락으로부터 추측하는 것입니다.

Skip-gram 모델

Skip-gram은 **중심 단어로부터 주변 단어들을 예측**하는 모델입니다.



한국어 예시:

- 입력: "**정말**" (중심 단어)
- 출력: "고양이는", "매우", "귀엽다"

수학적 표현:

```
P(w_{t-c}, ..., w_{t-1}, w_{t+1}, ..., w_{t+c} | w_t)
```

직관적 설명: Skip-gram은 연상 게임과 같습니다. "정말"이라는 단어를 보고 함께 나올 수 있는 주변 단어들을 모두 떠올리는 것입니다.

모델 비교 (Model Comparison)

특성 (Feature)	CBOW	Skip-gram
예측 방향 (Prediction Direction)	맥락 → 중심	중심 → 맥락
학습 속도 (Training Speed)	빠름 (Fast)	느림 (Slow)
소량 데이터 (Small Data)	우수 (Good)	보통 (Average)
희귀 단어 (Rare Words)	보통 (Average)	우수 (Good)
계산 복잡도 (Computational Complexity)	낮음 (Low)	높음 (High)
사용 권장 (Recommended Use)	빈번한 단어	희귀한 단어

한국어 선택 가이드:

- **CBOW 선택**: 한국어 뉴스 기사처럼 일반적인 단어가 많은 대용량 데이터
- **Skip-gram 선택**: 한국어 전문 용어나 고유명사가 많은 특수 도메인 데이터

상세한 아키텍처 비교 (Detailed Architecture Comparison)

CBOW 아키텍처

```
입력 (Input): [w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}]

↓ (원-핫 인코딩, One-hot Encoding)
```

```
입력층 (Input Layer): [V × 4] (V: 어휘 크기, Vocabulary Size)

↓ (가중치 행렬 W_in, Weight Matrix)

은닉층 (Hidden Layer): [N] (N: 임베딩 차원, Embedding Dimension) - 평균화

↓ (가중치 행렬 W_out, Output Weight Matrix)

출력층 (Output Layer): [V] (소프트맥스, Softmax)

↓

예측 (Prediction): P(w_t | context)
```

Skip-gram 아키텍처

```
입력 (Input): w_t

↓ (원-핫 인코딩, One-hot Encoding)

입력층 (Input Layer): [V]

↓ (가중치 행렬 W_in, Input Weight Matrix)

은닉층 (Hidden Layer): [N]

↓ (가중치 행렬 W_out, Output Weight Matrix)

출력층 (Output Layer): [V × 2c] (각 맥락 위치별, For each context position)

↓

예측 (Prediction): P(w_{t+j} | w_t) for j ∈ [-c, c], j ≠ 0
```

계층적 소프트맥스 (Hierarchical Softmax)

기본 소프트맥스의 문제점 (Problems with Standard Softmax)

일반적인 소프트맥스 함수는 다음과 같이 정의됩니다:

```
P(w_o | w_i) = exp(v'_{w_o} · v_{w_i}) / Σ_{w=1}^V exp(v'_w · v_{w_i})
```

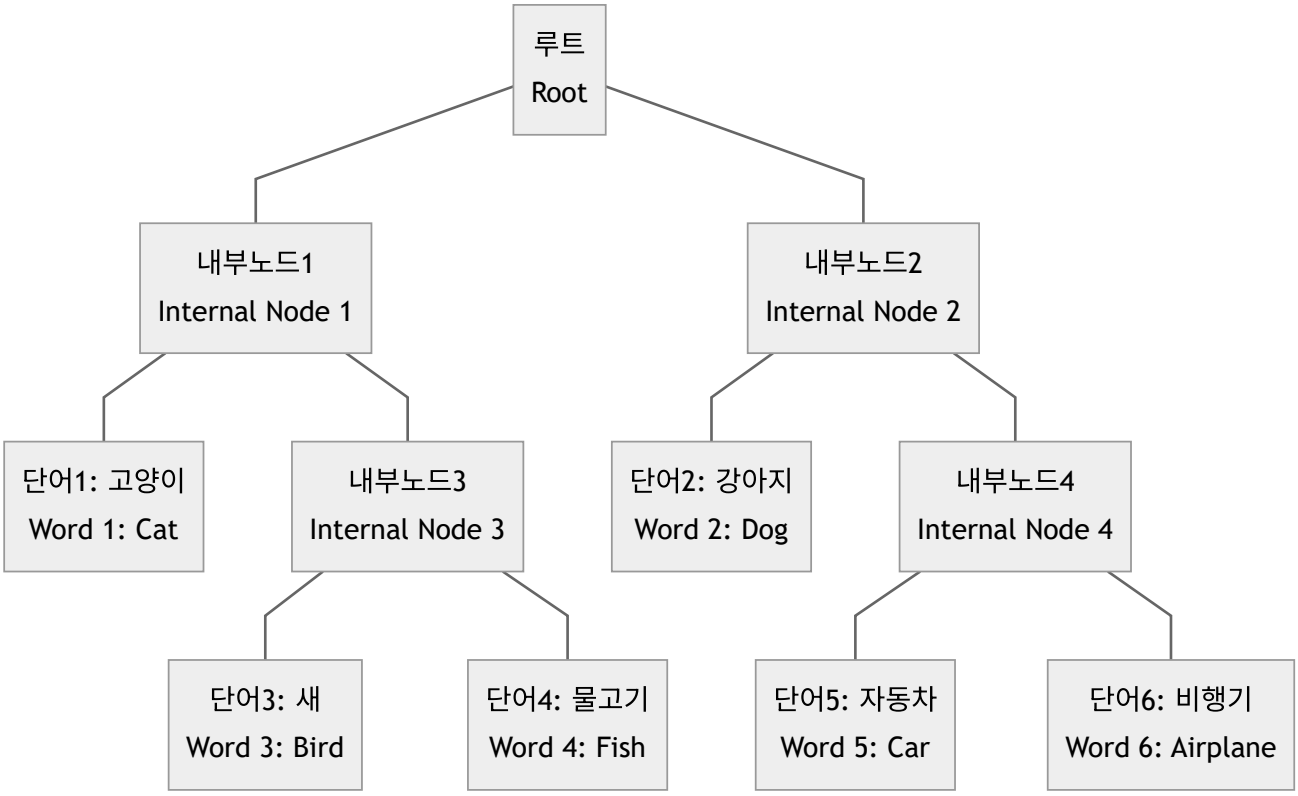
문제점:

- 분모의 정규화 항목이 전체 어휘 V에 대해 계산되어야 함
- 시간 복잡도: O(V) - 어휘 크기에 비례
- 대용량 어휘(수십만~수백만 단어)에서 계산 비용이 매우 큼

한국어 비유: 마치 전교생 중에서 1등을 뽑기 위해 모든 학생의 점수를 다 계산해야 하는 것과 같습니다. 학생 수가 많을수록 계산이 오래 걸립니다.

계층적 소프트맥스의 해결책 (Hierarchical Softmax Solution)

계층적 소프트맥스(Hierarchical Softmax)는 **이진 트리 구조(Binary Tree Structure)**를 사용하여 계산 복잡도를 O(log V)로 줄입니다.



한국어 비유: 이제 토너먼트 방식으로 1등을 뽑습니다. 전교생을 반으로 나누고, 또 반으로 나누어 단계별로 경쟁시키면 훨씬 빠르게 결과를 얻을 수 있습니다.

수학적 원리 (Mathematical Principle)

각 내부 노드는 **이진 분류기(Binary Classifier)**로 작동합니다:

```
P(left | node, context) = σ(v'_{node} · h)

P(right | node, context) = 1 - σ(v'_{node} · h)
```

여기서:

- `σ(x) = 1/(1 + exp(-x))` : 시그모이드 함수 (Sigmoid Function)
- `v'_{node}` : 내부 노드의 벡터 (Internal Node Vector)
- `h` : 은닉층 출력 (Hidden Layer Output, 맥락 벡터)

단어 확률 계산 (Word Probability Calculation)

특정 단어 w의 확률은 **루트에서 해당 단어까지의 경로(Path)**를 따라 계산됩니다:

```
P(w | context) = Π_{j=1}^{L(w)-1} P(n(w,j+1) | n(w,j), context)
```

여기서:

- `L(w)` : 단어 w까지의 경로 길이 (Path Length)
- `n(w,j)` : 경로상의 j번째 노드 (j-th Node in Path)
- `P(n(w,j+1) | n(w,j), context)` : 노드 간 전이 확률 (Transition Probability)

한국어 예시 계산

"고양이" 단어의 확률 계산:

```
P(고양이 | context) = P(left | 루트) × P(left | 내부노드1)

= σ(v'_{루트} · h) × σ(v'_{내부노드1} · h)
```

직관적 설명: "고양이"를 찾기 위해 루트에서 시작해서 “왼쪽으로 가기” → "또 왼쪽으로 가기"의 확률을 곱한 것입니다.

허프만 트리 최적화 (Huffman Tree Optimization)

word2vec은 **허프만 코딩(Huffman Coding)**을 사용하여 트리를 구성합니다:

1. **빈도 기반 구성**: 자주 나오는 단어일수록 루트에 가깝게 배치
2. **경로 길이 최소화**: 평균 경로 길이를 최소화하여 계산 효율성 극대화
3. **균형 트리**: 가능한 한 균형잡힌 이진 트리 구성

한국어 비유: 자주 사용하는 물건을 손이 닿기 쉬운 곳에 두는 것과 같습니다. “은, 는, 이, 가” 같은 조사는 트리의 위쪽에, “헬리콥터” 같은 희귀한 단어는 아래쪽에 배치합니다.

장점:

- 시간 복잡도: O(V) → O(log V)
- 메모리 효율성: 내부 노드만 추가 벡터 저장
- 빈도 기반 최적화: 자주 사용되는 단어의 빠른 계산

네거티브 샘플링 (Negative Sampling)

기본 개념 (Basic Concept)

네거티브 샘플링(Negative Sampling)은 계층적 소프트맥스의 대안으로, **소수의 네거티브 샘플(Negative Samples)**만을 사용하여 학습 효율성을 높이는 기법입니다.

한국어 비유: 모든 학생과 비교하는 대신, 몇 명의 "가짜 정답"과만 비교해서 진짜 정답을 구별하는 방법입니다. 훨씬 빠르면서도 효과적입니다.

수학적 원리 (Mathematical Principle)

기존 소프트맥스 목적함수를 다음과 같이 근사합니다:

원래 목적함수 (Original Objective):

```
log P(w_o | w_i) = log σ(v'_{w_o} · v_{w_i}) - log Σ_{w=1}^V exp(v'_w · v_{w_i})
```

네거티브 샘플링 목적함수 (Negative Sampling Objective):

```
log σ(v'_{w_o} · v_{w_i}) + Σ_{j=1}^k E_{w_j ~ P_n(w)} [log σ(-v'_{w_j} · v_{w_i
```

여기서:

- `w_o` : 타겟 단어 (positive sample, 정답 샘플)
- `w_j` : 네거티브 샘플들 (negative samples, 오답 샘플들)
- `k` : 네거티브 샘플 개수 (보통 5-20개)
- `P_n(w)` : 네거티브 샘플링 분포 (Negative Sampling Distribution)

네거티브 샘플링 분포 (Negative Sampling Distribution)

word2vec에서는 다음 분포를 사용합니다:

```
P_n(w) = f(w)^{3/4} / Σ_{w'} f(w')^{3/4}
```

여기서 `f(w)` 는 단어 w의 빈도입니다.

3/4 지수의 의미:

- **지수 1**: 빈도에 비례 (빈번한 단어만 선택)
- **지수 0**: 균등 분포 (모든 단어 동일 확률)

- **지수 3/4**: 빈번한 단어와 희귀한 단어의 균형

한국어 예시: "은, 는, 이, 가" 같은 조사는 너무 자주 나오고, "헬리콥터" 같은 단어는 너무 드뭅니다. 3/4 지수를 사용하면 둘 다 적절히 샘플링됩니다.

알고리즘 과정 (Algorithm Process)

```
graph TD

A[중심 단어 w_i<br/>Center Word] --> B[타겟 단어 w_o 선택<br/>Select Target Word]

B --> C[Positive 샘플<br/>σ(v'_{w_o} · v_{w_i})]

A --> D[k개 네거티브 샘플 선택<br/>Select k Negative Samples]

D --> E[Negative 샘플들<br/>σ(-v'_{w_j} · v_{w_i})]

C --> F[손실 함수 계산<br/>Calculate Loss Function]

E --> F

F --> G[그래디언트 업데이트<br/>Gradient Update]
```

상세한 학습 과정 (Detailed Learning Process)

1. **Positive 샘플 처리 (Positive Sample Processing):**

```
L_pos = log σ(v'_{w_o} · v_{w_i})
```

- 타겟 단어의 확률을 최대화 (정답을 맞히도록 학습)

2. **Negative 샘플 처리 (Negative Sample Processing):**

```
L_neg = Σ_{j=1}^k log σ(-v'_{w_j} · v_{w_i})
```

- 네거티브 샘플들의 확률을 최소화 (오답을 틀리도록 학습)

3. **전체 손실 함수 (Total Loss Function):**

```
L = L_pos + L_neg
```

한국어 직관적 설명: "고양이"라는 단어 주변에 "귀여운"이 나올 확률은 높이고, 무작위로 선택한 “자동차”, “수학” 같은 단어들이 나올 확률은 낮추는 방식으로 학습합니다.

그래디언트 계산 (Gradient Calculation)

Positive 샘플에 대한 그래디언트:

```
∂L/∂v_{w_i} = (1 - σ(v'_{w_o} · v_{w_i})) · v'_{w_o}

∂L/∂v'_{w_o} = (1 - σ(v'_{w_o} · v_{w_i})) · v_{w_i}
```

Negative 샘플에 대한 그래디언트:

```
∂L/∂v_{w_i} = -σ(v'_{w_j} · v_{w_i}) · v'_{w_j}

∂L/∂v'_{w_j} = -σ(v'_{w_j} · v_{w_i}) · v_{w_i}
```

네거티브 샘플링의 장점 (Advantages of Negative Sampling)

1. **계산 효율성**: O(V) → O(k), 여기서 k << V
2. **메모리 효율성**: 전체 어휘에 대한 소프트맥스 계산 불필요
3. **품질 유지**: 적은 샘플로도 좋은 성능 달성
4. **구현 단순성**: 계층적 소프트맥스보다 구현이 간단

하이퍼파라미터 설정 (Hyperparameter Settings)

| 데이터 크기 (Data Size) | 네거티브 샘플 수 (k) |

|----------|----------|

| 소규모 데이터셋 (Small Dataset) | 5-20 |

| 대규모 데이터셋 (Large Dataset) | 2-5 |

한국어 경험적 규칙:

- **작은 데이터셋**: 한국어 소설 몇 권 → 더 많은 네거티브 샘플 필요 (10-20개)
- **큰 데이터셋**: 한국어 위키피디아 전체 → 적은 네거티브 샘플로도 충분 (2-5개)

벡터 공간의 의미적 관계 (Semantic Relationships in Vector Space)

코사인 유사도 (Cosine Similarity)

word2vec에서 단어 간 유사도는 **코사인 유사도(Cosine Similarity)**로 측정됩니다:

```
similarity(w1, w2) = cos(θ) = (v_{w1} · v_{w2}) / (||v_{w1}|| × ||v_{w2}||)
```

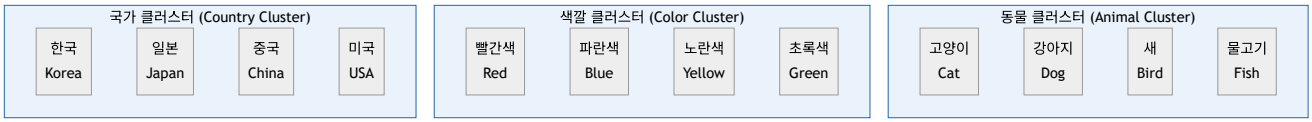
여기서:

- `v_{w1}, v_{w2}` : 단어 w1, w2의 벡터
- `·` : 내적 (dot product, 점곱)
- `||v||` : 벡터의 크기 (L2 norm, 유클리드 노름)

한국어 비유: 두 사람의 성격이 얼마나 비슷한지 측정하는 것과 같습니다. 각 사람을 여러 특성(활발함, 친절함, 창의성 등)으로 표현하고, 이 특성들이 얼마나 비슷한 방향을 가리키는지 계산합니다.

의미적 클러스터링 (Semantic Clustering)

비슷한 의미를 가진 단어들은 벡터 공간에서 **클러스터(Cluster)**를 형성합니다:



실제 한국어 예시 (Real Korean Examples)

한국어 word2vec 모델에서의 유사도 예시:

```
# "고양이"와 유사한 단어들 (Similar words to "cat")
model.most_similar("고양이", topn=5)
# 결과 예시:
# [("강아지", 0.85), ("새", 0.72), ("토끼", 0.68), ("물고기", 0.65), ("햄스터", 0.62
```

```
# "서울"과 유사한 단어들 (Similar words to "Seoul")
model.most_similar("서울", topn=5)
# 결과 예시:
# [("부산", 0.78), ("대구", 0.75), ("인천", 0.72), ("광주", 0.69), ("대전", 0.67)]
```

벡터 공간의 기하학적 특성 (Geometric Properties of Vector Space)

1. **거리와 유사도의 관계 (Distance-Similarity Relationship):**

```
거리가 가까울수록 → 코사인 유사도가 높음
거리가 멀수록 → 코사인 유사도가 낮음
```

2. **차원별 의미 해석 (Dimensional Meaning Interpretation):**

- 각 차원은 특정 의미적 특성을 나타냄
- 예: 차원 1 = "생물/무생물", 차원 2 = "크기", 차원 3 = "감정" 등

3. **선형 관계 (Linear Relationships):**

- 의미적으로 관련된 단어들은 선형적 관계를 보임
- 이는 아날로지 연산의 기반이 됨

한국어 직관적 설명: 벡터 공간을 3차원 방으로 생각해보세요. 비슷한 의미의 단어들은 같은 구석에 모여있고, 관련된 단어들은 일정한 패턴으로 배치되어 있습니다.

다차원 공간에서의 시각화 (Visualization in High-dimensional Space)

고차원 벡터를 2D/3D로 시각화하기 위해 **t-SNE** 또는 **PCA**를 사용:

```
from sklearn.manifold import TSNE # t-SNE 라이브러리
import matplotlib.pyplot as plt # 시각화 라이브러리


# word2vec 벡터를 2D로 축소 (Dimensionality reduction to 2D)
tsne = TSNE(n_components=2, random_state=42)
vectors_2d = tsne.fit_transform(word_vectors)


# 시각화 (Visualization)
plt.scatter(vectors_2d[:, 0], vectors_2d[:, 1])
for i, word in  enumerate(words):
plt.annotate(word, (vectors_2d[i, 0], vectors_2d[i, 1]))
```

의미적 관계의 종류 (Types of Semantic Relationships)

1. **동의어 관계 (Synonymous Relationship):**

```
"크다" ≈ "거대하다" ≈ "큰"

유사도: 0.8 ~ 0.9
```

2. **상하위 관계 (Hypernym-Hyponym Relationship):**

```
"동물" → "고양이" (상위 → 하위)

유사도: 0.6 ~ 0.8
```

3. **연관 관계 (Associative Relationship):**

```
"의사" ↔ "병원" (직업 ↔ 장소)

유사도: 0.5 ~ 0.7
```

4. **반의어 관계 (Antonymous Relationship):**

```
"크다" ↔ "작다"

유사도: 0.3 ~ 0.5 (맥락이 비슷하므로 완전히 0이 아님)
```

한국어 특별한 관계들:

- **높임말 관계**: “먹다” ↔ “드시다” (유사도: 0.7~0.8)
- **한자어-순우리말**: “산” ↔ “뫼” (유사도: 0.6~0.7)
- **지역 방언**: “감자” ↔ “지슬” (제주 방언, 유사도: 0.5~0.6)

아날로지와 벡터 연산 (Analogy and Vector Operations)

아날로지의 기본 원리 (Basic Principle of Analogy)

word2vec의 가장 놀라운 특성 중 하나는 **벡터 산술(Vector Arithmetic)**을 통한 아날로지 해결 능력입니다.

기본 공식:

```
vec("왕") - vec("남자") + vec("여자") ≈ vec("여왕")
```

이는 다음과 같이 해석됩니다:

- "왕"에서 "남성성"을 빼고 "여성성"을 더하면 "여왕"이 됨

한국어 직관적 설명: 마치 수학 공식처럼 의미를 계산할 수 있습니다. "왕 - 남자 + 여자 = 여왕"은 "권력자에서 성별만 바꾸면?"이라는 질문에 대한 답을 구하는 것입니다.

수학적 원리 (Mathematical Principle)

아날로지 연산 `A : B = C : D` 는 다음과 같이 계산됩니다:

```
D* = argmax_{w∈V} similarity(vec(B) - vec(A) + vec(C), vec(w))
```

여기서:

- `D*` : 예측된 답 (Predicted Answer)
- `similarity()` : 코사인 유사도 함수 (Cosine Similarity Function)
- `V` : 전체 어휘집합 (Entire Vocabulary Set)
- `argmax` : 최댓값을 가지는 인수를 찾는 함수

다양한 한국어 아날로지 유형 (Various Korean Analogy Types)

1. 성별 관계 (Gender Relationships)

```
남자 : 여자 = 왕 : ?

vec("여자") - vec("남자") + vec("왕") ≈ vec("여왕")


아버지 : 어머니 = 아들 : ?

vec("어머니") - vec("아버지") + vec("아들") ≈ vec("딸")


형 : 누나 = 남동생 : ?

vec("누나") - vec("형") + vec("남동생") ≈ vec("여동생")
```

2. 지리적 관계 (Geographical Relationships)

```
서울 : 한국 = 도쿄 : ?

vec("한국") - vec("서울") + vec("도쿄") ≈ vec("일본")


부산 : 바다 = 대전 : ?

vec("바다") - vec("부산") + vec("대전") ≈ vec("내륙")


경복궁 : 서울 = 불국사 : ?

vec("서울") - vec("경복궁") + vec("불국사") ≈ vec("경주")
```

3. 한국어 시제 관계 (Korean Tense Relationships)

```
가다 : 갔다 = 오다 : ?

vec("갔다") - vec("가다") + vec("오다") ≈ vec("왔다")


먹다 : 먹었다 = 자다 : ?

vec("먹었다") - vec("먹다") + vec("자다") ≈ vec("잤다")


하다 : 했다 = 보다 : ?

vec("했다") - vec("하다") + vec("보다") ≈ vec("봤다")
```

4. 한국어 높임법 관계 (Korean Honorific Relationships)

```
먹다 : 드시다 = 자다 : ?

vec("드시다") - vec("먹다") + vec("자다") ≈ vec("주무시다")


말하다 : 말씀하시다 = 묻다 : ?

vec("말씀하시다") - vec("말하다") + vec("묻다") ≈ vec("여쭙다")
```

5. 복수형 관계 (Plural Relationships)

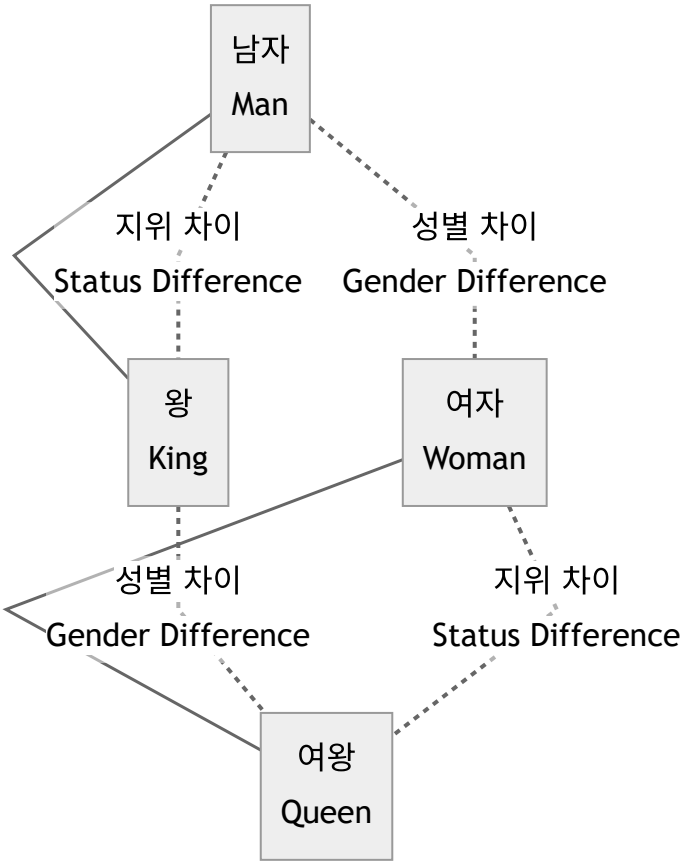
```
cat : cats = dog : ?

vec("cats") - vec("cat") + vec("dog") ≈ vec("dogs")


child : children = man : ?
```

```
vec("children") - vec("child") + vec("man") ≈ vec("men")
```

벡터 공간에서의 기하학적 해석 (Geometric Interpretation in Vector Space)



이 관계는 벡터 공간에서 **평행사변형(Parallelogram)**을 형성합니다:

- 수평 벡터: 성별 차이 (Gender Difference)
- 수직 벡터: 지위 차이 (Status Difference)

한국어 직관적 설명: 마치 지도에서 방향을 찾는 것과 같습니다. "남자에서 왕으로 가는 방향"과 "여자에서 여왕으로 가는 방향"이 평행하다는 것을 발견한 것입니다.

실제 구현 예시 (Practical Implementation Example)

```
def  analogy(model, word1, word2, word3, topn=1):
"""
아날로지 연산 함수 (Analogy operation function)
```

```
word1 : word2 = word3 : ?
"""
# 벡터 연산 수행 (Perform vector operation)
result_vector = model[word2] - model[word1] + model[word3]
# 가장 유사한 단어 찾기 (Find most similar words)
similar_words = model.similar_by_vector(result_vector, topn=topn+3)
# 입력 단어들 제외 (Exclude input words)
filtered_results = []
for word, similarity in similar_words:
if word not in [word1, word2, word3]:
filtered_results.append((word, similarity))
if  len(filtered_results) >= topn:
break
return filtered_results


# 한국어 사용 예시 (Korean usage example)
result = analogy(model, "남자", "왕", "여자")
print(result) # [("여왕", 0.85)]


# 지역 관계 예시 (Regional relationship example)
result = analogy(model, "서울", "한국", "도쿄")
print(result) # [("일본", 0.82)]
```

아날로지 성능 평가 (Analogy Performance Evaluation)

아날로지 성능은 다음 데이터셋으로 평가됩니다:

1. **Google Analogy Dataset:**

- 19,544개의 아날로지 문제
- 의미적/구문적 관계 포함

2. **평가 지표 (Evaluation Metrics):**

```
정확도 = 정답 개수 / 전체 문제 수
```

3. **한국어 아날로지 예시 (Korean Analogy Examples):**

```
서울:한국 = 파리:프랑스 (지리적 관계)

크다:큰 = 작다:작은 (형용사 활용)

먹다:먹는다 = 가다:간다 (동사 활용)

아버지:어머니 = 아들:딸 (가족 관계)
```

아날로지의 한계 (Limitations of Analogy)

1. **문맥 의존성 (Context Dependency):**

- 단어의 다의성으로 인한 오류
- 예: “배”(과일) vs “배”(교통수단) vs “배”(신체부위)

2. **문화적 편향 (Cultural Bias):**

- 학습 데이터의 편향이 결과에 반영
- 예: 성별, 직업, 지역 관련 편향

3. **복잡한 관계 (Complex Relationships):**

- 단순한 선형 관계로 표현하기 어려운 복잡한 의미 관계
- 예: "사랑"과 "미움"의 관계는 단순한 반대가 아님

한국어 특별한 한계들:

- **동음이의어**: “눈”(eye) vs “눈”(snow)
- **높임법 복잡성**: 상황에 따른 다양한 높임 표현
- **지역 방언**: 표준어와 방언 간의 미묘한 차이

벡터 연산의 확장 (Extensions of Vector Operations)

아날로지 외에도 다양한 벡터 연산이 가능합니다:

```
# 의미적 중심점 찾기 (Finding semantic center)
center = (vec("고양이") + vec("강아지") + vec("새")) / 3
# 결과: "동물"과 유사한 벡터


# 의미적 차이 계산 (Calculating semantic difference)
difference = vec("왕") - vec("여왕")
# 결과: "성별 차이"를 나타내는 벡터


# 클러스터 중심 이동 (Moving cluster center)
new_vector = vec("의사") + (vec("병원") - vec("학교"))
# 결과: "의사"를 "병원" 맥락으로 이동


# 한국어 특별한 연산들 (Special Korean operations)
honorific_diff = vec("드시다") - vec("먹다") # 높임법 차이
```

```
new_honorific = vec("가다") + honorific_diff # "가시다" 예측


regional_diff = vec("서울말") - vec("표준어") # 지역 차이

busan_style = vec("부산말") + regional_diff # 부산 방언 특성
```

이러한 벡터 연산들은 word2vec이 단순한 단어 표현을 넘어서 **의미적 관계를 수치적으로 조작(Numerical Manipulation of Semantic Relationships)**할 수 있음을 보여줍니다.

결론 (Conclusion)

word2vec은 분포 가설(Distributional Hypothesis)을 기반으로 단어를 밀집 벡터(Dense Vector)로 표현하는 혁신적인 기법입니다. Skip-gram과 CBOW 두 가지 아키텍처를 통해 효율적인 학습이 가능하며, 계층적 소프트맥스(Hierarchical Softmax)와 네거티브 샘플링(Negative Sampling)을 통해 대규모 어휘에서도 실용적인 성능을 달성합니다.

특히 벡터 공간에서의 의미적 관계 표현과 아날로지 연산 능력은 자연어 처리 분야에 큰 영향을 미쳤으며, 현재까지도 많은 NLP 애플리케이션의 기초 기술로 활용되고 있습니다.

한국어 사용자를 위한 핵심 포인트:

1. **이해하기 쉬운 비유**: 복잡한 수학적 개념을 일상적인 예시로 설명
2. **실용적인 활용**: 한국어 특성을 고려한 구체적인 사용 사례 제시
3. **기술적 정확성**: 영문 원어와 한글 설명을 병기하여 정확한 이해 도모

이 문서에서 설명한 원리들을 바탕으로, 다음 단계에서는 실제 구현체의 코드 구조와 세부 알고리즘을 분석해보겠습니다.